Computer Science

Instructor's Edition

Table of Contents

1. Instructor's Guide 5
   1.1. Who this course is for 5
   1.2. What is inside 5
   1.3. Before you assign exercises 5
   1.4. Assesment 5
   1.5. Using and adapting this book 6
2. Computer Architecture 7
   2.1. Introduction 7
   2.2. CPU 7
      2.2.1. ALU 7
      2.2.2. CU 7
      2.2.3. CPU Characteristics 7
   2.3. Memory 7
      2.3.1. RAM 8
      2.3.2. HDD 8
   2.4. Converting data sizes (Decimal System - SI) 8
   2.5. Number Systems 9
      2.5.1. Base-10 9
      2.5.2. Base-2 9
      2.5.3. Relationship between base-2 and base-10 9
         2.5.3.1. Converting from Base-10 to Base-2 9
         2.5.3.2. Converting from Base-2 to Base-10 9
   2.6. Logic gates 10
      2.6.1. Introduction 10
      2.6.2. NOT 10
      2.6.3. AND 10
      2.6.4. NAND 10
         2.6.4.1. De Morgan's Law 11
      2.6.5. OR 11
      2.6.6. NOR 11
         2.6.6.1. De Morgan's Law 11
      2.6.7. XOR 11
3. Python 13
   3.1. Introduction 13
   3.2. Data Types 14
   3.3. Variables 14
   3.4. Operators 14
      3.4.1. Arithmetic operators 14
      3.4.2. Comparison operators 15
      3.4.3. Logical Operators On A Truth Table 15
      3.4.4. Membership Operators 15
      3.4.5. Bitshift operators 15
      3.4.6. Operation + Assignment By Examples 16
   3.5. Loops 16
      3.5.1. A visual 16

3.5.2. Example: Using lists to match two n-length lists 17
3.5.3. Using lists without the range function 17
3.6. Functions 17
3.6.1. A visual 18
3.6.2. Parameters & Arguments 18
3.6.3. Returning Values 18
3.6.4. Deterministic & Pure function 18
3.6.5. Using functions to calculate the hypotenuse of a right triangle ABC 18
3.6.6. Functions to be aware of 19
3.6.6.1. print 19
3.6.6.2. int 19
3.6.6.3. range 19
3.7. Control Flow 19
3.7.1. A visual 19
3.7.2. If statements 19
3.7.3. Else statements 20
3.7.4. Using control flow to categorize age 20
3.8. Try it yourself 20
4. Complexity Theory 21
4.1. What is an algorithm? 21
4.1.1. Example: Sequential Search 21
4.1.2. Example: Binary Search 22
4.2. Time vs Space complexity? 23
4.2.1. Big-O complexity graph 23
4.2.2. Definition 23
4.2.2.1. Explanation 23
4.2.3. Ways of proving f(n) = O(g(n)) 25
4.2.4. Why is knowing the time complexity of an algorithm important? 26
5. Miscellaneous 27
5.1. Cache 27
5.1.1. The client-server model 27
5.1.1.1. The client 27
5.1.1.2. The server 27
5.1.1.3. Illustration 28
5.1.1.4. The caching of client-server models 28
5.1.2. In the context of a CPU 29
5.1.2.1. CPU cache levels 30
5.1.3. Summary 30
6. Exam studying methodology 30
7. Exercises 31
7.1. Computer Architecture 31
7.2. Logic Gates 33
7.3. Solve Operations: 33
7.4. Problem: When is $x \vee y = c$ solvable? 33
7.5. Python 34
8. Exams 46
8.1. Section 1 : Choose the correct answer (6 points) 46
8.1.1. 1-) Random Access Memory is volatile storage because: 46
8.1.2. 2-) What is the primary role of the Central Processing Unit: 46

8.1.3. 3-) Python is interpreted because: 46
8.2. Section 2 (3 points): 46
8.3. Section 3: Identify the pieces of code (3 points) 46
8.4. Section 4: From The Text (8 points) 47
8.5. Section 5: Bonus 47

1. Instructor's Guide

1.1. Who this course is for

This is a first course in computer science for students in grades 10-12. It assumes no prior exposure and only basic algebra and some calculus, the material is deliberately simplified in places (The CPU is just an ALU and a CU, "interpreted" as line-by-line execution). Refine these when your class is ready.

1.2. What is inside

Part 2 covers computer architecture, number systems, logic gates. Part 3 covers Python fundementals. Part 4 covers complexity theory. Part 5 is a short unit on caching.

After those come the exercises, sample exam.

Core	Optional (stronger classes or extra time)
CPU, ALU, CU, memory types, and units	Logic gates
Binary and Decimal	Bitshift operations
Python types, operators, loops, functions, control flow	Pure functions, deterministic functions
	Complexity Theory and Big-O
	Caching
	Monte Carlo Exercises

In my first run with a beginner class, I left out complexity, logic gates, caching, entirely, the rest in the optional category was not examined.

1.3. Before you assign exercises

Some exercisews rely on Python the notes don't introduce: building and updating dictionaries, `len()`, and importing `random` and `math`. Teach these first. Complexity Theory also benifts from an introduction to logarithms, inequalities, and how functions grow.

1.4. Assesment

Start slow: Open with easy muiltiple-choice quiz on architecture, definitions, and hold logic gates until later, you can also make text-comprehension of computer-science books and blogs, include few open-ended questions. **Add difficulty gradually**: move through code reading, short calculations, and proofs. **Always include definition questions**: so every student has attainable marks. **Bonus question**: include a few per test to recover lost marks, final grade is capped at 20/20.

1.5. Using and adapting this book

The source and all PDFs live at https://github.com/MMinusOne/CNDK-Computer-Science-Classes, the source is written in Typst, and its documentation is at https://typst.app/docs/. Expect a short learning curve before editing, and adjust the content to your course.

2. Computer Architecture

2.1. Introduction

Computer architecture is the structure of a computer system made from component parts.

2.2. CPU

- The CPU (**C**entral-**P**rocessing-**U**nit).
- It consists of many smaller units that handle different things, but in the explanation we will take, it will only contain an ALU 2.2.1 and a CU 2.2.2, both of their explanations will be simplified.
- It's essentially the brain of the computer, handling all the computation (we can call them tasks) that happen on the computer.



Figure 1: A physical CPU

,

2.2.1. ALU

- The ALU (**A**rithmetic-**L**ogic-**U**nit) is a component/unit of the CPU 2.2
- It handles all the arithmetic (addition, subtraction, division, multiplication, etc...)
- All the operations that happen inside the ALU are in base-2

2.2.2. CU

- The CU (**C**ontrol-**U**nit) is a component/unit of the CPU 2.2
- It will manage and execute instructions.
- For now, we will think of instructions as operations needed to perform certain actions on the computer (adding two numbers, opening an app, etc...)

2.2.3. CPU Characteristics

- A CPU has a clock speed: the speed at which cycles will execute, and it is a frequency.
- $1\text{GHz} = 1 * 10^9$ Hz

2.3. Memory

> Memory in the context of a computer system is the place where "data" will be stored. Data can be for example: images, text, files, Variables 3.3, etc...

We will discuss:

- Two types of memory: Primary Memory and Secondary Memory
- Data units: bit, byte, KB, MB, GB

Primary Memory	Secondary Memory
Fast access speed	Slower access speed
Volatile (data lost on restart)	Non-volatile (data persists)
Example: **RAM**	Example: **HDD, SSD**
Used for active tasks	Used for long-term storage

2.3.1. RAM

- RAM (**R**andom-**A**ccess-**M**emory) is a type of memory.
- RAM is primary memory.

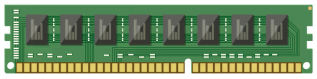


Figure 2: A physical stick of RAM

,

2.3.2. HDD

- HDD (Hard-Disk-Drive) is a type of memory.
- HDD is secondary-memory.



Figure 3: An HDD

2.4. Converting data sizes (Decimal System - SI)

Unit	Conversion
Bit	Smallest data unit
Byte	1 byte = 8 bits
Kilobyte (KB)	1 KB = 1,000 bytes
Megabyte (MB)	1 MB = 1,000 KB
Gigabyte (GB)	1 GB = 1,000 MB

2.5. Number Systems

2.5.1. Base-10

- Base-10 is the number system we humans use.
- It's made up of 10 digits, digits from (0 to 9): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Any other number is made up from these `10` digits.

2.5.2. Base-2

- Base-2 is the number system computers use.
- It's made up of two digits: `0` and `1`.
- Any other number is made up from these `2` digits.

2.5.3. Relationship between base-2 and base-10

Numbers can be converted to-and-back from base-2 to base-10.

2.5.3.1. Converting from Base-10 to Base-2

Example: Convert $(75)_{10}$ to binary

Step 1: Make a table.

Step 2: Divide 75 by 2 repeatedly.

Step 3: If the number is a decimal, take everything before the decimal and assign it the bit `1`; if the number is a whole number, write the number as is and assign it the bit `0`.

Step 4: Construct the number by reading the bits from bottom to top.

Step 5: If the number is now less than 1, record it, and stop.

number	index	bit
37	0	1
18	1	1
9	2	0
4	3	1
2	4	0
1	5	0
$\frac{1}{2}$	6	1

Table 1: Table for converting 75 to base-2.

2.5.3.2. Converting from Base-2 to Base-10

We will take the same example as above and in Table 1, and we will use the indexes to remake the base-10 number.

we had $(75)_{10} = (1001011)_2$

We need to only consider the "1"s and their indexes as shown in Figure 4

$(1001011)_2 = (2^0 + 2^1 + 2^3 + 2^6)_{10} = (75)_{10}$

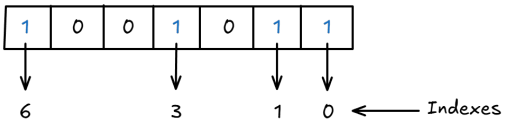


Figure 4: The 1's and their indexes.

2.6. Logic gates

2.6.1. Introduction

A logic gate is a device that performs a boolean function (logical operation). It is the smallest unit of computation in a CPU 2.2.

In this section, we will think of each logic gate as a device that has one or two inputs (NOT has one, all the others have two) and one output.

The primitive logic gates are: NOT, AND, OR.

From these we can build the derived gates: NAND, NOR, XOR. NAND and NOR are also called *universal* gates, because any other gate can be built using only NAND gates (or only NOR gates).

2.6.2. NOT

A NOT gate can be represented by the following table:

I_1	O_1
0	1
1	0

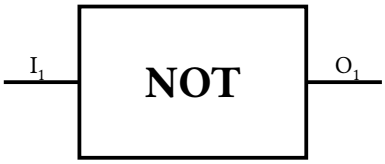


Figure 5: NOT

- The **NOT** gate outputs the complement of A, meaning 0 if A is 1, and 1 if A is 0.
- We will denote the operation: $\neg A = O_1$
- We can also denote the operation with a bar: $\overline{A} = O_1$

2.6.3. AND

An AND gate can be represented by the following table:

I_1	I_2	O_1
0	0	0
0	1	0
1	0	0
1	1	1



Figure 6: AND

- The **AND** gate outputs 0 for all pairs of (A, B), except when both **A** AND **B** are 1.
- We will denote the operation: $A \wedge B = O_1$

2.6.4. NAND

A NAND gate can be represented by the following table:

I_1	I_2	O_1
0	0	1
0	1	1
1	0	1
1	1	0



Figure 7: NAND

- The **NAND** gate outputs the opposite of what an AND gate would produce for a pair (A, B).
- We will denote the operation: $\neg(A \wedge B) = O_1$

2.6.4.1. De Morgan's Law

$X = \neg(A \wedge B) \equiv \neg A \vee \neg B$

2.6.5. OR

An OR gate can be represented by the following table:

I_1	I_2	O_1
0	0	0
0	1	1
1	0	1
1	1	1

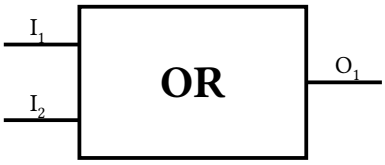


Figure 8: OR

- The **OR** gate outputs 1 if **A** OR **B** (or both) is 1, and 0 only when both are 0.
- We will denote the operation: $A \vee B = O_1$

2.6.6. NOR

A NOR gate can be represented by the following table:

I_1	I_2	O_1
0	0	1
0	1	0
1	0	0
1	1	0



Figure 9: NOR

- The **NOR** gate outputs the opposite of what an OR gate would produce for a pair (A, B), meaning it outputs 1 only when both **A** and **B** are 0.
- We will denote the operation: $\neg(A \vee B) = O_1$

2.6.6.1. De Morgan's Law

$X = \neg(A \vee B) \equiv \neg A \wedge \neg B$

2.6.7. XOR

An XOR (exclusive OR) gate can be represented by the following table:

I_1	I_2	O_1
0	0	0
0	1	1
1	0	1
1	1	0



Figure 10: XOR

- The **XOR** gate outputs 1 when **A** and **B** are different, and 0 when they are the same. Unlike OR, it outputs 0 when both inputs are 1.
- We will denote the operation: $A \oplus B = O_1$

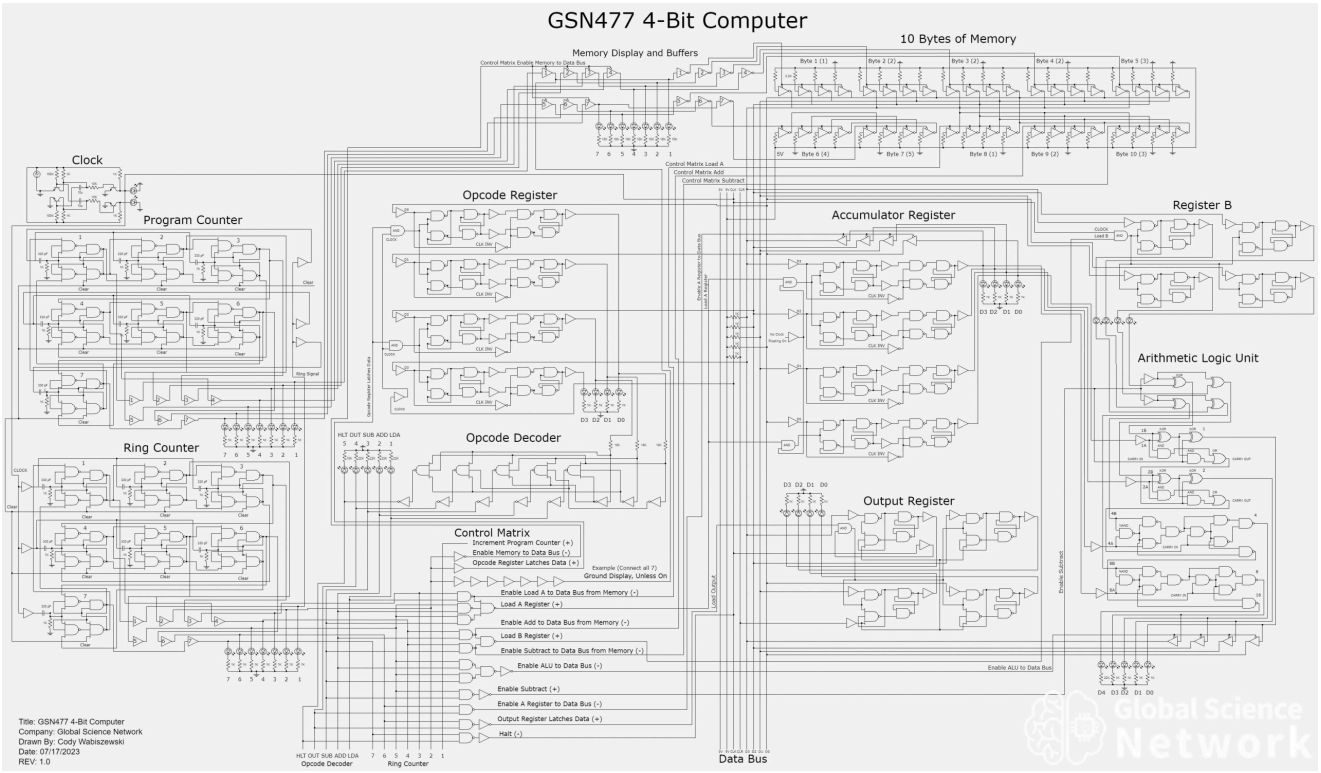


Figure 11: Image of a 4-bit Computer built with logic-gates

3. Python

3.1. Introduction

> **Definition: Python is a “high-level”, “interpreted”, “general-purpose” scripting language.**

Characteristic	Explanation
High-Level	Built on many lower-level abstractions. Python provides an easy-to-use language by building upon harder programming concepts and technologies.
Interpreted	An external program (the interpreter) reads and executes instructions line by line. Unlike compiled languages, Python code is interpreted at runtime.
General-Purpose	Can be used for most use-cases. Python is versatile: web development, data science, automation, etc.
Sequential	Executes instructions from top to bottom by default.
Imperative	Uses imperative programming style to maintain state.

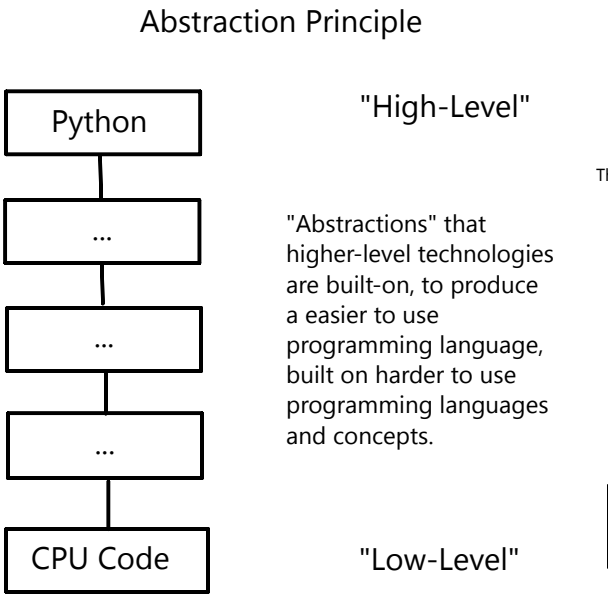


Figure 12: Compiled languages process.

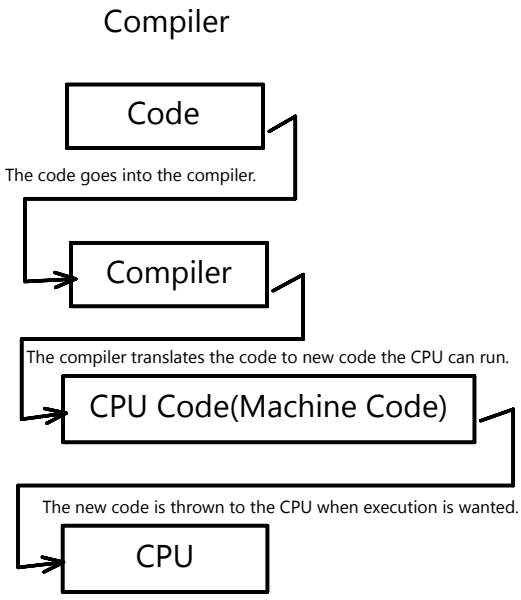


Figure 13: Abstractions.

3.2. Data Types

Type	Description & Examples
`int`	Integer - a whole number. Examples: `10`, `25`, `300`, `-5`
`float`	Decimal number. Examples: `52.6`, `19.3`, `1.0`, `5.0`
`bool`	Boolean - either `True` or `False`
`str`	String - sequence of characters. Examples: `"Hello"`, `"Text"`, `""`
`list`	Sequence of elements. Example: `[10, 5.2, 9.8]` Lists are heterogeneous and 0-indexed
`dict`	Dictionary - key-value store. Example: `{"name": "John", "age": 30}` Access: `person["name"]` ⟶ `"John"`
`None`	Represents a null value

3.3. Variables

Variables in Python are **mutable** (can change) and **dynamically typed** (no specific type restriction).

examples:

```
name = "Johnny"
johnny_age = 20
grades = [10, 5.2, 9.8, 12.5, 13.2]
person = {"name": "Johnny", "age": johnny_age}
```

3.4. Operators

3.4.1. Arithmetic operators

Operator	Name	Example
+	Addition	`10 + 5 = 15`
-	Subtraction	`10 - 5 = 5`
*****	Multiplication	`10 * 5 = 50`
/	Division	`10 / 5 = 2.0`
******	Exponentiation	`3 ** 4 = 3 * 3 * 3 * 3 = 81`

Important: Division always returns a float: `int / int → float`

3.4.2. Comparison operators

Operator	Name	Result
==	Equality	`10 == 10` ⟶ `True`
!=	Inequality	`10 != 5` ⟶ `True`
>	Greater than	`10 > 5` ⟶ `True`
<	Less than	`10 < 5` ⟶ `False`
>=	Greater or equal	`10 >= 10` ⟶ `True`
<=	Less or equal	`5 <= 10` ⟶ `True`

3.4.3. Logical Operators On A Truth Table

p	q	Conjunction p **and** q $p \wedge q$	Disjunction p **or** q $p \vee q$	Negation **not** p $\neg p$	Negation **not** q $\neg q$
True	True	True	True	False	False
True	False	False	True	False	True
False	True	False	True	True	False
False	False	False	False	True	True

3.4.4. Membership Operators

Operator	Explanation	Example

`in`	Used to check if a certain key is inside a dictionary, returns a bool	`"name" in {"name": "John"}` ⟶ `True`
		because key "name" exists in the dict
		`"age" in {"name": "John"}` ⟶ `False`
		because key "age" does not exist in the dict

`not in`	Used to check if a certain key is not inside a dictionary, returns a bool	`"name" not in {"name": "John"}` ⟶ `False`
		because key "name" exists in the dict
		`"age" not in {"name": "John"}` ⟶ `True`
		because key "age" does not exist in the dict

3.4.5. Bitshift operators

- 9 .) Left-bitshift (<<): `x << n`, where `x` and `n` are `int`s, will shift the bits of `x` by `n` zeroes and return the base-10 `int`.

Example:

- `5 << 2`:
- We convert 5 to base-2: $(5)_{10} = (101)_2$.
- And left shift by 2: `(10100)` (We added two zeroes.)
- Now if we convert back to base-10, the number becomes: $2^2 + 2^4 = 20$.

- 10 .) Right-bitshift (>>): The opposite procedure of left-bitshift.

Example:
- `5 >> 2`
- If we convert 5 to base-2: $(5)_{10} = (101)_2$.
- And right shift by 2: `(001)`, (The `01` that was there moved to somewhere that doesn't exist.)
- Now we convert back to base-10, the number becomes: $2^0 = 1$.

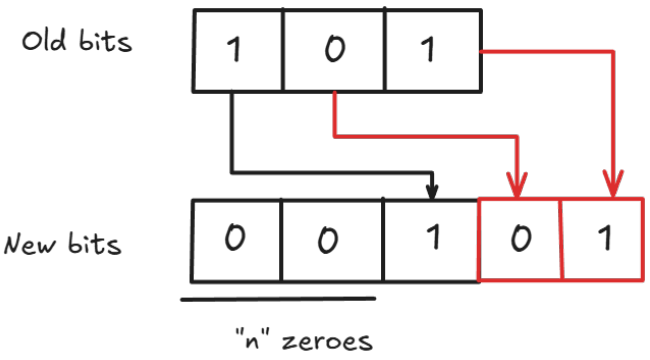


Figure 14: Right bitshift

3.4.6. Operation + Assignment By Examples

All the arithmetic, and bitshift operators have an assignment equivalent used to modify variables.

```
# We will declare these variables for example purposes
first_number = 10
second_number = 5
first_string = "John"
second_string = "Johnny"

first_string += " Jason" # first_string will be "John Jason" after that, because "John"
+ " Jason" = "John Jason".
second_number -= 6 # second_number will be -1 after that, because 5 - 6 = -1.
second_number *= -2 # second_number will be 2 after that, because (-1)(-2) = 2.
first_number /= 5 # first_number will be 2.0 after that, because 10 / 5 = 2.0.
first_number = 5 # first_number will be 5 after that, normal assignment.
first_number <<= 2 # first_number will be 20 after that, because 5 << 2 = 20
first_number >>= 3 # first_number will be 2 after that, because 20 >> 3 is 2.
```

3.5. Loops

Loops are used for repeated actions.

3.5.1. A visual

```
accumulator = 0 # A variable that accumulates

for current_number in range(0, 10): # A loop over the range 0 to 10 (10 exclusive), and
current_number is the current value from the range
  accumulator += current_number # Mutating accumulator and adding current_number to it
```

After this, `accumulator` will be $0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$.

3.5.2. Example: Using lists to match two n-length lists

```
names  = ["John", "Johnnathan", "Jay", "Jane", "Jake"] # List of n names
grades = [9, 7.5, 6, 3.5, 6] # List of n grades
n      = 5

for i in range(0, n): # Looping n times
  name = names[i] # Accessing the i-th name from the names list
  grade = str(grades[i]) # Accessing the i-th grade from the grades list, and converting
it to a string
  print(name + " has a grade of " + grade) # Concatenating (Adding) the strings together
to make a sentence, here we see why we turned grade into a "str", because str + int and
str + float is not possible
```

The output is as shown below:

```
John has a grade of 9
Johnnathan has a grade of 7.5
Jay has a grade of 6
Jane has a grade of 3.5
Jake has a grade of 6
```

3.5.3. Using lists without the range function

Until now, we are still using the `range` function, but are **iterable** by default. An iterator allows any object to be **iterated over** by a loop. Let's compare:

```
nums = [1,2,3,4,5]
n = 5

for i in range(0, n):
  print(nums[i])
```

Code 1: Looping over lists with range

```
nums = [1,2,3,4,5]

for num in nums:
  print(num)
```

Code 2: Looping over lists with the list's iterative property

3.6. Functions

Functions should be thought of as machines that take an input and give us an output.



Figure 15: Function diagram

3.6.1. A visual

```
def add(x, y):
  return x + y
```

Code 3: A function that adds two numbers, x and y.

Now, to use this function we just **defined**, we will need to **call** it, as such:

```
result = add(10, 5)
print(result)
```

Code 4: Calling the function add

The output of result will now be `15`, since the function handled the addition of `x` and `y`, and `x` was `10`, and `y` was `5`.

3.6.2. Parameters & Arguments

- The parameters of a function is what we label the inputs. Consider Code 3, the parameters of that function are `x` and `y`.
- The arguments of a function is what is supplied for the labels of the inputs. Consider Code 4, the arguments are `10` and `5`.

3.6.3. Returning Values

The **return value** of a function is what the function outputs. Taking as example Code 3, the return value can be described as `x + y`, or also the sum of its inputs.

The return value is also what comes after the **return** keyword.

3.6.4. Deterministic & Pure function

Function Characteristic	Definition
Deterministic Function	A deterministic function is a function that returns the same outputs for the same inputs, and has no randomness in the determination of future states.
Pure Function	A pure function is a function that obeys two properties: • It is deterministic • The function has no side effect, meaning no mutation of non-local variables.

3.6.5. Using functions to calculate the hypotenuse of a right triangle ABC

```
def calculate_hypotenuse(AB, AC):
  BC = (AB**2 + AC ** 2) ** 0.5
  return BC
```

We can describe this function as follows:

- This function is called `calculate_hypotenuse`.
- It takes two parameters: AB and AC, which are supposed to be either ints or floats.
- It returns the hypotenuse of the triangle ABC.

3.6.6. Functions to be aware of

3.6.6.1. print

Output to the standard output (simplified).

```
>>> print("Hello World")
--> Hello World
```

3.6.6.2. int

> **Returns an integer constructed from a number or a string.**

```
int(123.45) # 123
int("123") # 123
int("01110011", base=2) # 115
```

3.6.6.3. range

Range is technically a *class* not a function, but we will refer to it as a function. We will consider valid arguments to be integers.

Refer to Loops 3.5 to know how to use it.

3.7. Control Flow

Control flow describes how execution progresses from one command to the next.

3.7.1. A visual

```
your_age = 15
adult_age = 18

if your_age >= adult_age:
  print("You are an adult")
else:
  print("You are a minor")
```

Code 5: Control flow, if and else.

In the code above, we have two variables, `your_age` and `adult_age`.

We first compare `your_age` and `adult_age` using `>=` (Bigger than or equal to), which returns a bool type, this bool type is then considered by the **if** statement before it, and accordingly, the **branch** below if (the truthy-case branch) will be executed, otherwise, we are handling it with an **else** statement.

3.7.2. If statements

if executes it's branch if the statement given to it is a truthy value, a **truthy** value is a value that is not **falsy**.

Some **falsy** values in Python are: `None`, `0` and `False`

truthy values evaluate to **True** booleans, and **falsy** values evaluate to **False** booleans.

The structure of an **if statement** looks as such:

```
if boolean:
  branch instructions
```

3.7.3. Else statements

else statements are chained with **if** statements, and are executed in case the previous **if statement** turns out to be falsy, the structure is as such:

```
if boolean:
  branch instructions
else:
  branch instructions
```

One thing possible with **else** statements is chaining them with if statements, as such:

```
if boolean:
  branch instructions
elif boolean:
  branch instructions
elif boolean:
  branch instructions
else:
  branch instructions
```

Here, we have an if statement, chained with else statements, and a default case.

3.7.4. Using control flow to categorize age

```
age = 30

if age >= 60:
    print("You're a senior!")
elif age >= 40:
    print("You're an adult!")
elif age >= 20:
    print("You're a young adult!")
elif age >= 13:
    print("You're a teenager!")
elif age >= 3:
    print("You're a child!")
else:
    print("You're an infant!")
```

3.8. Try it yourself

You can go to the website: https://www.programiz.com/python-programming/online-compiler/, and follow everything each step of the way.

4. Complexity Theory

Complexity theory studies the relationship between an algorithm and the resource it takes.

4.1. What is an algorithm?

A computer algorithm is a sequence of steps a computer will take to achieve a particular outcome.

4.1.1. Example: Sequential Search

Given a sorted array A of length n, $(A_0 \leq A_1 \leq A_2 \leq ... \leq A_{n-1})$, and a number K, $K \in A$ sequential search is an algorithm that will go through each index in the array one by one, starting with 0, and stops when it finds the element K in the array and return it's position.

For example, if I have the array $A = [1, 5, 9, 12, 60, 110, 120, 121, 500, 505]$ and the number $K = 60$, sequential search will start checking in order:

- Is $A_0 = 60$? No, keep going.
- Is $A_1 = 60$? No, keep going.

...

- Is $A_4 = 60$? Yes, return 4, the desired position.
- This algorithm has a time complexity of $O(n)$, where $n = \text{length of the array}$, we will explain why later, but intuitively, we can think it is $O(n)$ because if the number K was at the end of the array(worst case scenario), it will take n steps to find K

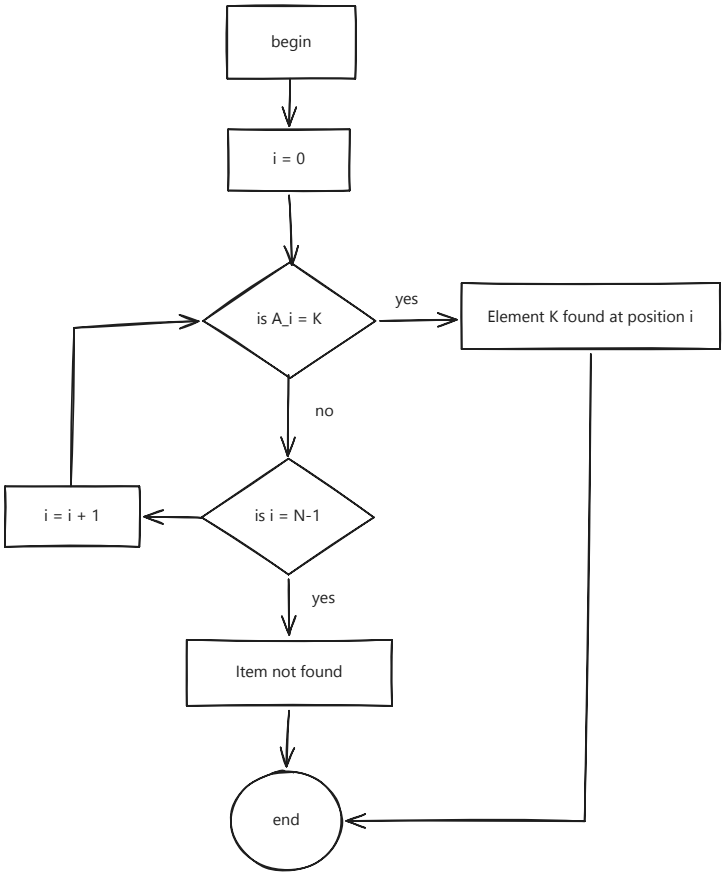


Figure 16: Sequential Search Diagram

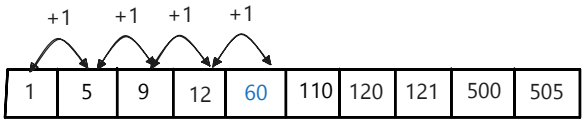


Figure 17: Sequential Search Array

4.1.2. Example: Binary Search

For binary search, we will consider the **sorted** 10-length array $A = [-5, -2, 0, 1, 2, 4, 5, 6, 7, 10]$ and $K = 7$, and we will start with low = 0, and high = 9, at each point, we will then repeatedly take midpoint $=$ low $+ \left\lfloor \frac{\text{high} - \text{low}}{2} \right\rfloor$, and call that point M, if $A_m > K$, then K is positioned before position m (the array is sorted), so we will take the left half, otherwise if $A_m < K$, then K is positioned after the position m, so we will take the right half, and repeat.

- This algorithm has $O(\log_2 n)$, but is written as simply $O(\log n)$ because 2 is a constant term we don't usually consider, this is because at each step in the algorithm, we cut the amount of work we have to do by half.

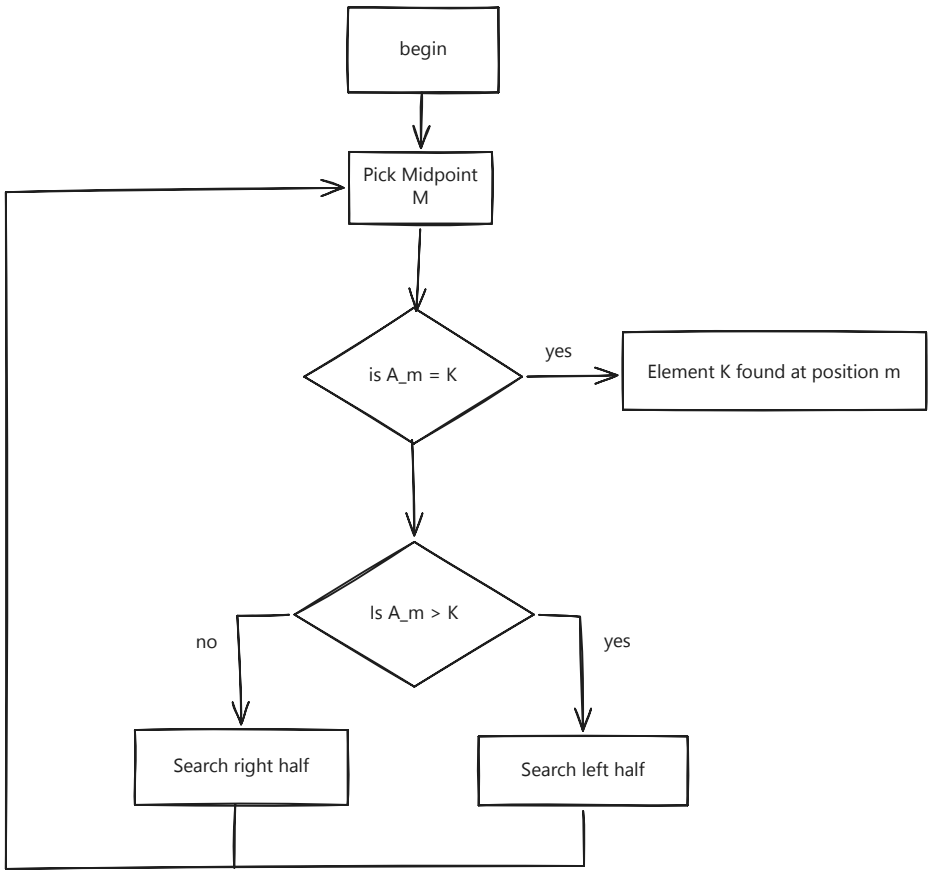


Figure 18: Binary Search Diagram

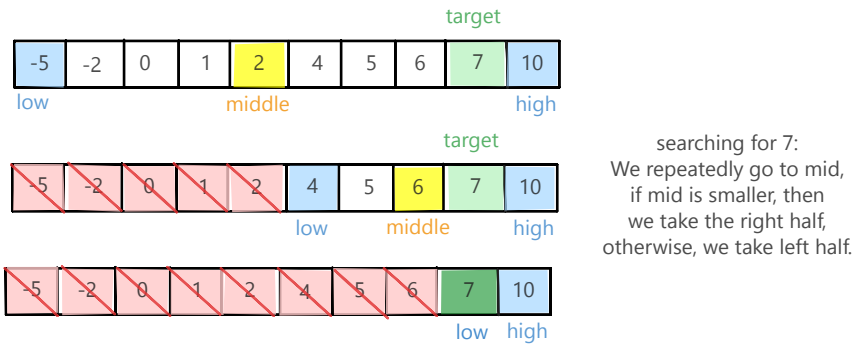


Figure 19: Binary Search Array

4.2. Time vs Space complexity?

- Time complexity describes how the execution time(number of steps) of an algorithm grows as the size of it's input grows.
- Space complexity describes how much more space an algorithm will take as the size of it's input grows.

In this course we will only be interested in time complexity.

4.2.1. Big-O complexity graph

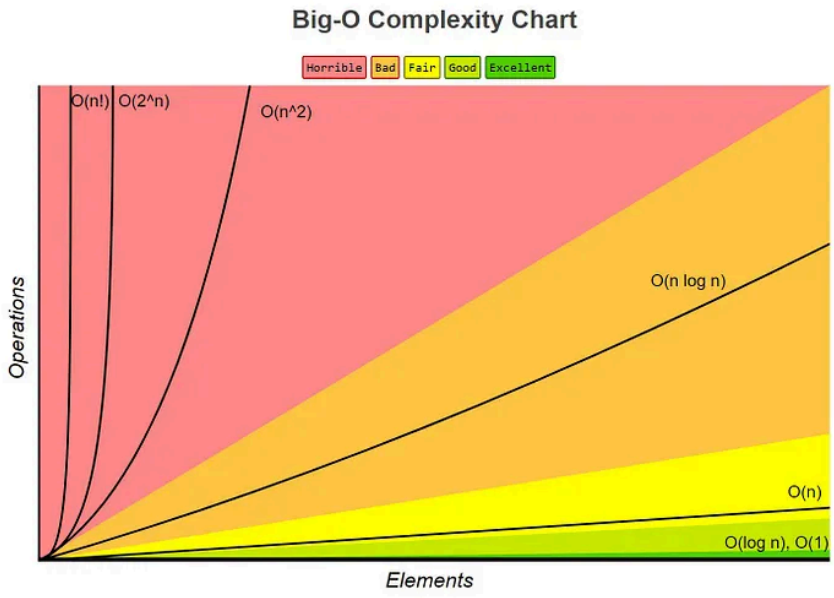


Figure 20: Big-O Complexity Chart

These are the standard time complexities we usually encounter, but we will only talk about $O(1)$, $O(n)$ and $O(n^2)$, it is important to note here that $n \geq 0$, and no negative input size exists.

4.2.2. Definition

$$f(n) = O(g(n)) \Leftrightarrow \exists c > 0, \exists n_0 > 0, \text{such that}$$
$$f(n) \leq c * g(n) \quad \forall n \geq n_0$$

$f(n)$ is $O(g(n))$ if there exists a $c > 0$ and an $n_0 > 0$, such that $f(n) \leq c * g(n)$ for all values of $n \geq n_0$

4.2.2.1. Explanation

Big-O describes the "upper-bound" of time an algorithm can take. Therefore, Big-O has $f(n)$ always at most $c * g(n)$ for all $n \geq n_0$.

There exists other complexity-notations:

Notation	Symbol	Meaning
Big-O	O	Upper Bound
Little-o	o	Strict upper bound
Big-Omega	Ω	Lower bound
Little-omega	ω	Strict lower bound
Big-Theta	Θ	Upper + Lower bound

The following would be true for Big-O, but not for little-o:

$$x^2 \in O(x^2)$$

$$x^2 \in O(x^2 + x)$$

$$x^2 \in O(200x^2)$$

We will only be responsible for **Big-O** in this course.

A tip: You can look at the highest degree of the function, and already be aware of what time complexity it is .

4.2.3. Ways of proving f(n) = O(g(n))

Substitution	Rebuilding
• If we want to prove $f(n) = n^2 + 5n + 3 = O(n^2)$ • We identify $g(n) = n^2$ We will need, by definition: $n^2 + 5n + 3 \leq c * g(n)$ $n^2 + 5n + 3 \leq c * n^2$ • For all $n \geq 1$, we have: $n \leq n^2$ $5n \leq 5n^2$ $3 \leq 3n^2$ • And since these are all bigger than the terms in $f(n)$, we can say: $n^2 + 5n + 3 \leq n^2 + 5n^2 + 3n^2$ $n^2 + 5n + 3 \leq 9n^2$ • So using $c = 9$ and $n_0 = 1$, we got a solution.	• If we want to prove $f(n) = n^2 + 5n + 3 = O(n^2)$ • We will take: $n_0 = 5$ $n \geq n_0$ $n \geq 5$ $5 \leq n$ • Multiplying by n, we get the middle term: $5n \leq n^2$ • For the constant term, since $n \geq 5$ we have $3 \leq n$, and $n \leq n^2$, so: $3 \leq n^2$ • And we replace: $n^2 + 5n + 3 \leq n^2 + n^2 + n^2$ $n^2 + 5n + 3 \leq 3n^2$ So using $c = 3$ and $n_0 = 5$, we got a solution.

4.2.4. Why is knowing the time complexity of an algorithm important?

Let's imagine a sorted array A like we did before, and let's say that array has a length of $N = 10^9$ (1 billion elements).

Using **sequential search** $O(n)$, would mean an upper-bound of operations $= o = 10^9$ operations (billion operations), to estimate how long that takes, we count CPU cycles per micro-operation:

Operation	Theoretical CPU Cycles
Load $A[i]$ from memory	1
Compare $A[i] == K$	1
Branch on result (if true stop, if false continue)	1
Increment i	1
Check if $i < n$ and branch	1

These are $c \approx 5$ cycles, and the time for the n operations in relation with clock speed and 5-cycles:

$$T(o) = \frac{\text{cycle count} * \text{operation count}}{\text{Clock Speed Frequency}} = \frac{5o}{f}$$

Where f is the clock-speed of the CPU (the speed of operation cycling) in "Hz". and o is the operation count. If we consider a theoretical CPU of clock speed 0.295 GHz (Same speed as a PlayStation 2).

$$T(o) = T(10^9) = \frac{5(10^9)}{(0.295)(10^9)} \approx 16.94 \text{ s}$$

It would take 16.94 seconds on a 0.295GHz CPU to execute the algorithm.

On the other hand, if we considered **binary search** instead with: $N = 10^9$, we would have an upper bound of:

$$o' = \log_2 N = \log_2 10^9 \approx 30$$

And so:

$$T(o') = T(\log_2 N) = \frac{5(\log_2 10^9)}{(0.295)(10^9)} \approx 5 * 10^{-7} \text{ s} \approx 0.0000005 \text{ s}$$

Comparing the two times: $\frac{T(o)}{T(o')} = \frac{16.94}{5*10^{-7}} \approx 33880000$, that's 33 million times faster. Now, realistically, there is one more operation cycle binary search does (calculating the midpoint), but I have omitted that in this calculation. This improvement in runtime cost is an **asymptotic improvement**

If I were to consider the actual CPU cycles per operations of binary search, the result would be like:

$$T_{\text{seq}}(o) = \frac{5o}{f} \qquad T_{\text{bin}}(o) = \frac{6o}{f}$$

$T_{\text{seq}}(o) = T_{\text{seq}}(10^9) = \frac{5(10^9)}{(0.295)(10^9)} \approx 16.94 \text{ s}$ $T_{\text{bin}}(o') = T_{\text{bin}}(\log_2 N) = \frac{6(\log_2 10^9)}{(0.295)(10^9)} \approx 0.0000006 \text{ s}$

(T_{bin} has 6 cycles per operation, one extra for calculating the mid-point)

Which still, isn't a big difference from the original estimation.

5. Miscellaneous

5.1. Cache

A cache is a place of storage of temporary information that get added(**inserted**) and removed (**evicted**) based on certain criteria, the criteria to add information in the cache is called an **insertion policy**, and the policy to remove something from the cache is called an **eviction policy**, this mechanism is mainly used for performance reasons.

- **TikTok example scenario** : A TikTok video is stored in the US, and you amongst other Lebanese people watch that video really often on their phones, TikTok might consider that fact, and decide to insert the video in a cache (the placement of storage) that is closer to Lebanon, that way when people want to go watch that video, it goes and gets that video information from somewhere closer, resulting in faster speeds.

5.1.1. The client-server model

The client server model is made up of two components, the: the client, and the server, and focuses on the relationship and communication that happens between these two components.

5.1.1.1. The client

The client (also the consumer in this relationship), will consume information, this can be for example: you watching TikTok on your phone, the client is your phone.

5.1.1.2. The server

The server (the producer in this relationship), will produce and send information to the client, in the example of TikTok, the TikTok **servers** will send out the video information (title, description, comments, likes, reposts, video buffer, etc...) to your phone.

5.1.1.3. Illustration



Figure 22: The client-server model

5.1.1.4. The caching of client-server models

Usually caching in the client-server model is based on geographical location, for this, you can imagine TikTok example scenario 21, let's first look at the figure Figure 23



Figure 23: Cache added on a standard client-server-model

Here, the TikTok system notices that multiple people, close to each other geographically, are watching the same video very frequently, so by nature, to satisfy those people's need for performance, the system decided to implement a cache, so instead of every time having to get the video information from New York, America, the information is gathered from Paris, France, resulting in a shorter response time, naturally data is only needed to be cached once, and is only cached again if it is evicted, so for example, after some amount of time, the system might decide: "There's no one watching this, let me free up space and free it from the cache."

5.1.2. In the context of a CPU

We can also apply the idea of a cache inside a CPU, we have said before that a Compiler 12 turns normal code into CPU code (more commonly called machine code), this machine will have to actually **consume** this code to do something meaningful with it, so a execution process without a cache might look like Figure 24, but with a cache implemented, might be like Figure 25, internally.

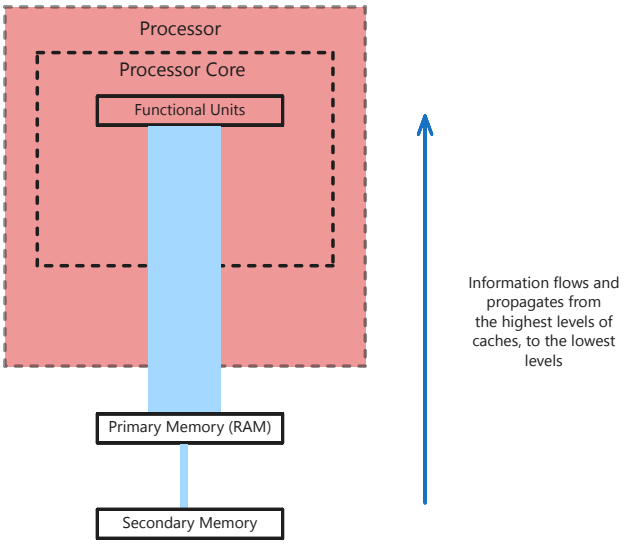


Figure 24: CPU without cache

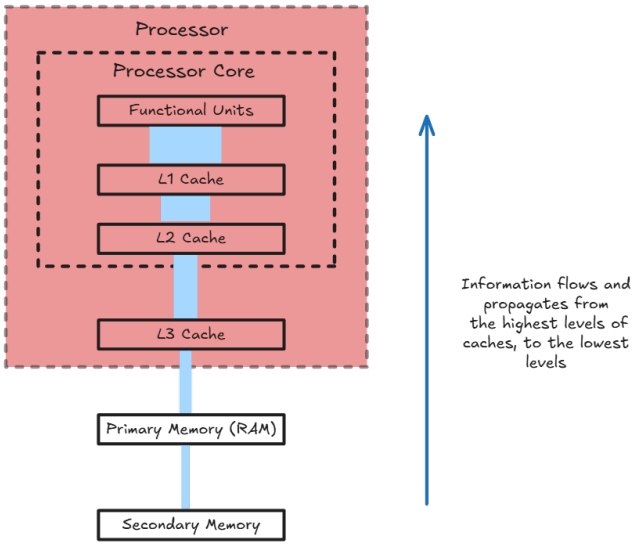


Figure 25: CPU with cache

5.1.2.1. CPU cache levels

In modern systems, usually CPUs have 3 cache levels, L1 cache, L2 cache and L3 cache, when the CPU can't find information in the L1 cache, it will go to the L2, if it can't find in the L2 it will go to L3, if it's not in the L3 it will go to primary memory, if it's not in primary memory it will go to secondary memory. When something isn't found in a particular cache, it is called a **cache miss**, otherwise it's a **cache hit**.

In summary, CPU caches improve performance, but add complexity on an existing system.

5.1.3. Summary

A cache is an Abstracted-Performance Layer 13 that is usually implemented between a producer and a consumer, a sender and a receiver, and emitter and a subscriber, that stores information temporary in a way that can be accessed faster for that consumer, such that there are performance gains.

6. Exam studying methodology

- Focus on understanding the concepts before memorizing them, there's not even the need to memorize them.
- 20% of the effort yields 80% of the outcome, don't try and nail all the details, just focus on the most important things.
- Focus on the English parts of things and try to explain things to yourself and other people to see if you understand.
- Regarding Python, you can try everything each step of the way, read Try it yourself 3.8

7. Exercises

7.1. Computer Architecture

EXERCISE: QUESTIONS ABOUT THE CPU

1. What two units we considered, exist inside the CPU?
2. What does “CPU” stand for?
3. What is it’s function?
4. If a CPU has a clock speed of 2.4GHz, calculate how many nanoseconds one clock cycle takes (Hint: $1\text{s} = 10^9\text{ns}$).

5. What is the function of the “ALU”?
6. What does ALU stand for?
7. Where is the ALU located?
8. What number system does the ALU understand?

9. What is the function of the “CU”?
10. Where is the CU located?
11. What does CU stand for?

SOLUTION:

1. We considered the ALU and the CU.
2. CPU stands for **C**entral **P**rocessing **U**nit.
3. It’s essentially the brain of the computer, handling all the computation.
4. We have $1\text{GHz} = 10^9\text{Hz}$:
   therefore $2.4\text{GHz} = 2.4 * 10^9\text{Hz}$
   Then we consider $T = \frac{1}{f}$
   So we get $T_c = \frac{1}{2.4*10^9}$ seconds, so now we have the time for one cycke T_c in seconds.
   Converting to nano seconds:

$$T_c = \frac{1}{2.4 * 10^9}\ \text{s} = \frac{1}{2.4 * 10^9} * 10^9\ \text{ns} = \frac{1}{2.4}\ \text{ns} \approx 0.417\text{ns}$$

5. It handles all the arithmetic (addition, subtraction, division, multiplication.
6. ALU stands for **A**rithmetic-**L**ogic-**U**nit.
7. The ALU is located in the CPU.
8. The number system used by the ALU is: base-2.

9. The CU manages and executes instructions.
10. The CU is located in the CPU.
11. CU stands for **C**ontrol-**U**nit.

EXERCISE: NUMBER SYSTEMS

1. Compare base-2 and base-10.
2. $(10011)_2 = (x)_{10}$, find x.
2. $(11011011011)_2 = (x)_{10}$, find x.
3. Convert $(1502)_{10}$, to base-2.
5. Convert $(1103)_{10}$ to base-2, and state how many bits are required to store the number.

SOLUTION:

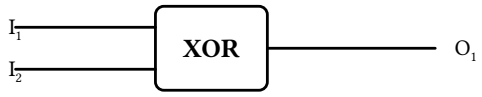
1. Base-2 is a number system made up of two digits 0 and 1, in contrast base-10 is made up from the 10 digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9).
2. We map each of the digits to it's index, and choose the 1's only, then calculate the sum using the method.
   $2^0 + 2^1 + 2^4 = 19$
3. $2^0 + 2^1 + 2^3 + 2^4 + 2^6 + 2^7 + 2^9 + 2^{10} = 1755$
4. $(1502)_{10} = (10111011110)_2$
5. $(1103)_{10} = (10001001111)_2$, and since the base-2 version has 11 bits, it will require 11 bits to store.

7.2. Logic Gates

Logic Gate Exercises

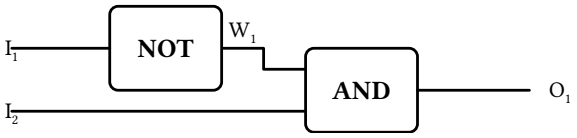
I = input, O = output, W = internal wire. Fill in the shaded cells for every combination of inputs.

Exercise 1



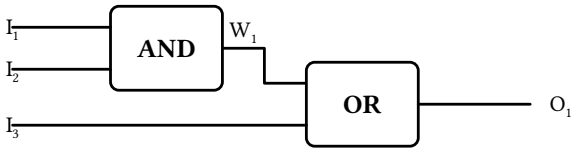
I_1	I_2	O_1
0	0	0
0	1	1
1	0	1
1	1	0

Exercise 2



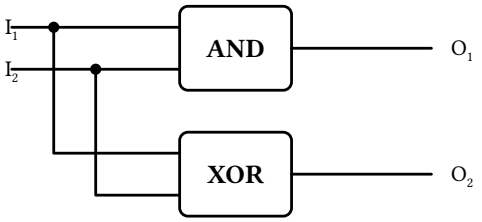
I_1	I_2	O_1
0	0	0
0	1	1
1	0	0
1	1	0

Exercise 3



I_1	I_2	I_3	O_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Exercise 4



I_1	I_2	O_1	O_2
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

7.3. Solve Operations:

- $\neg(100100)_2$
- $(101011)_2 \wedge (100100)_2$
- $(101011)_2 \vee (100100)_2$
- $(101011)_2 \oplus (100100)_2$

7.4. Problem: When is $x \vee y = c$ solvable?

You are given an arbitrary integer $x \in \mathbb{N}$, and an integer $c \in \mathbb{N}$. You can choose any $y \in \mathbb{N}$. Let:

- $a = (a_1, a_2, a_3, ..., a_{S-1}, a_S)$ be the list of bits of x
- $b = (b_1, b_2, b_3, ..., b_{S-1}, b_S)$ be the list of bits of c

- $\vee$ denote bit-wise OR on any two numbers.
- All numbers have the same bit-length, and S be the bit-length.
- a_i and b_i denote the bit at the i-th position ($1 \leq i \leq S$)
- $a_i, b_i \in \{0, 1\}$

What conditions must be satisfied at a_i and b_i such that: $x \vee y = c$?

7.5. Python

EXERCISE: DESCRIBE THE FOLLOWING FUNCTIONS

Function A:

```
def circle_area(radius):
  return 3.14 * (radius**2)
```

Function B:

```
def list_summation(num_list):
  summation = 0

  for num in num_list:
    summation += num

  return summation
```

Function C:

```
def smallest_in_list(num_list):
  smallest = None

  for num in num_list:
    if smallest == None:
      smallest = num
    elif num < smallest:
      smallest = num

  return smallest
```

1. Describe the functionality of each of these functions

EXERCISE: DESCRIBE THE FOLLOWING FUNCTIONS

Function D:

```
def solve_quadratic(a, b, c):
  delta = (b**2) - (4 * a * c)

  if delta >= 0:
```

```
    x_1 = (-b - delta**0.5) / (2 * a)
    x_2 = (-b + delta**0.5) / (2 * a)
    return [x_1, x_2]
  else:
    return "No real solutions"
```

Function E:

```
def vector(x_a, y_a, x_b, y_b):
  x_ab = x_b - x_a
  y_ab = y_b - y_a

  return [x_ab, y_ab]
```

1. Describe the functionality of each of these functions

SOLUTION:

1.

A)

- This function is called `circle_area`, and takes in a parameter `radius`.
- It uses the formula $A = \pi r^2$ to calculate the circle's area.
- Then it returns the area

B

- This function is called `list_summation`, and takes in a parameter `num_list`
- It then makes a variable that will hold the sum of all numbers in the list
- It then loops over each number in the list, adding it to that holder variable
- And finally, returns the sum

C:

- This function is called `smallest_in_list`, and accepts a parameter `num_list`
- It then creates a variable for the smallest number
- It then loops over each number in the list
  - If there is no smallest number yet, then the smallest number is the current number
  - If there is a smallest number, and it's bigger than the current number, than the current number is the new smallest number
- Then it returns the smallest number

D:

- This function is called `solve_quadratic` and accepts three parameters `a`, `b` and `c`.
- It then uses the formula $\Delta = b^2 - 4ac$ and holds the delta in a variable
  - If $\Delta \geq 0$, then it calculates the two (or double) roots of the equation and returns them in a list.
  - Else ($\Delta < 0$), it will return the string "No real solutions"

E:

- This function is called `vector` and takes in 4 parameters: `x_a`, `y_a`, `x_b`, `y_b`
- It will then calculate the difference between `x_a` and `x_b` and hold it in a variable

- It will also calculate the difference between `y_a` and `y_b` and hold it in a variable
- Finally, it returns the two variables in a list

EXERCISE: AN ALGORITHM THAT SATISFIES: NUMS[I] - 10 = K

Suppose we have a list `nums` and a constant k.

You must find the first index `i` such that `nums[i] - 10 == k`. If there is no such index that satisfies the property, return `None`.

Example cases:

Case 1:
- **Inputs**: nums = [5, 20, 35, 23, 2, 5, 30, 6, 2, 3], k = 20
- **Output**: 6
- **Explanation**: nums[6] is 30, 30 - 10 = 20, and k = 20.

Case 2:
- **Inputs**: nums = [5, 2, 60, 50, 20], k = 15
- **Output**: None
- **Explanation**: There is no index i such that `nums[i] - 10 = k`.

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - Then we need to loop the `range(n)` where n is the length of `nums`, while accessing the index `i`, and for each `nums[i] - 10`, see if it is `==` to `k`.
   - If an index is found, and the property is satisfied, we immediately return the index and exit the function.
   - Otherwise, we keep looping, and if the loop ends and no index is found, we return `None`.

   2.

```python
def solution(nums, k):
  for i in range(len(nums)):
    if nums[i] - 10 == k:
      return i

  return None
```

   3. The solution's time complexity is $O(n)$: Because the amount of instructions scale linearly ($y = ax + b$) with the input's size.
   The solution's space complexity is $O(1)$: Because we are only storing constant data regardless of input size.

EXERCISE: THE TWO SUM PROBLEM

Suppose we have a list `nums` and a constant `k`.

You must find two indices `i` and `j` such that `nums[i] + nums[j] = k` and $i \neq j$, if there are no such indices, return `None`.

Multiple solutions may be valid.

Example cases:

Case 1:
- **Inputs**: [3,2,4], k = 6
- **Outputs**: [1,2] or [2, 1]
- **Explanation**: nums[1] + nums[2] == 6, we return [1,2] (or [2,1])

Case 2:
- **Inputs**: nums = [2,7,11,15], k = 9
- **Outputs**: [0, 1] or [1, 0]
- **Explanation**: Because nums[0] + nums[1] == 9, we return [0, 1]

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - We will make a dict of numbers we have seen already, the keys will be numbers we've seen and the values will be indices.
   - We will loop over `nums`, accessing `i` and `nums[i]`.
   - Assuming we have `nums[i]`, `c = k - nums[i]` will be the number we need such that `nums[i] + c = k`, we will calculate that complement `c`.
   - If the complement is inside the dict, then we can return the index of the complement and `i`. (In other words, if the number c we need for `c + nums[i] = k` is inside the dict, we will return the indices of it and i).

2.
```
def solution(nums, k):
    seen = {}

    for i in range(len(nums)):
        num = nums[i]
        complement = k - nums[i]

        if complement in seen:
            j = seen[complement]
            return [i, j]

        seen[num] = i

    return None
```

3. The time complexity of the algorithm is: $O(n)$
   The space complexity of the algorithm is: $O(n)$

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - We will loop over `range(len(nums))` and access `i` and `nums[i]`.
   - For each `num` at `i`, we will loop over `range(i + 1, len(nums))`, to check the numbers in-front of `nums[i]` excluding `nums[i]` itself, from that loop we will access `j`.
   - In the previous loop, if `nums[i] + nums[j] == k`, then we can return `[i, j]`

2.
```python
def solution(nums, k):
  for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
      if nums[i] + nums[j] == k:
        return [i, j]

  return None
```

3. The time complexity of the algorithm is: $O(n^2)$: The function scales quadratically with its input.
   The space complexity of the algorithm is: $O(1)$: We are only storing constant data.

EXERCISE: APPROXIMATE PI USING MONTE CARLO METHODS

Monte Carlo methods are computational algorithms based on **repeated random sampling**, used to solve **deterministic** problems through **randomness**.

- **Area of a circle**: $a_c = \pi r^2$, where r is the radius.
- **Area of a square**: $a_t = a^2$, where a is the side-length of the square.
- **Equation of a circle**: $x^2 + y^2 = r^2$
- **Point $P(x_p, y_p)$ is inside the circle**: $x_p^2 + y_p^2 \leq r^2$
- **Point $P(x_p, y_p)$ is outside the circle**: $x_p^2 + y_p^2 > r^2$

Now, imagine a circle of radius r centered at $(0, 0)$, inscribed in a square, the square would have side-length $a = 2r$. The area of the square would then be $a_t = (2r)^2 = 4r^2$.

Now, imagine within the constraints of this square, we are randomly choosing n amount of points, and we will consider two cases:
- Points that landed in the circle
- Points that landed in the square

If you think about it, the proportion of points inside the circle approximates the proportion of the circle's area to the square's area. We will compute $\frac{a_c}{a_t}$ to get a formula for π.

$$\frac{a_c}{a_t} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Now, we will consider the amount of points that landed in the circle as the area of the circle (a_c), and the points that landed in the square as the area of the square (a_t), so we get:

$$\frac{\text{amount of points in the circle}}{\text{amount of points in the square}} \approx \frac{\pi}{4}$$

$$\pi \approx 4\left(\frac{\text{amount of points in the circle}}{\text{amount of points in the square}}\right)$$

The way we will resolve this equation using Monte Carlo is as following: for a number n, we will generate n points with random x and y coordinates, then, for each of these points, we will use the formulas to check if they landed in the circle or not, then we could resolve this equation.

Now, because `random.random()` only provides values between `0` and `1`, we will work with a quarter-circle (first-quadrant only) where: $0 \leq x \leq 1$ and $0 \leq y \leq 1$, which simplifies the process (no need to generate negative coordinates).

Here are the visuals: https://www.desmos.com/calculator/zmcv3rdppk

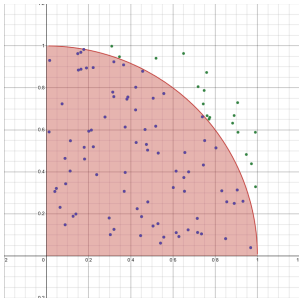


Figure 26: $n = 100$

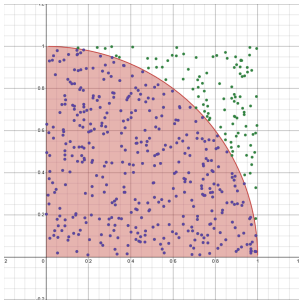


Figure 27: $n = 500$

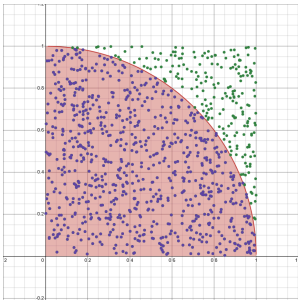


Figure 28: $n = 1000$

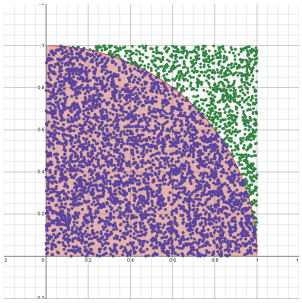


Figure 29: $n = 5000$

The purple points are the points *inside the circle*, and the green points are *outside the circle*.

1. Write a multi-step strategy towards solving the problem.
2. Write the algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that takes a parameter: `amount_of_points`, the number of points to be generated.
   - We will consider arbitrarily that $r = 1$ unit.
   - We will make a variable called `points_in_circle` to accumulate how many points landed in the circle.
   - We must loop `n` times.
   - For each loop, each time, we will generate a random `x_p` and `y_p` point that lies between 0 and 1 with `random.random()` from the `random` package.
   - We will then calculate `x_p**2 + y_p**2` for that point, and check if it is less than r^2, if it is, then it is inside the circle, and we will add `+1` point to `points_in_circle`.

2.
```
import random
```

```python
def solution(amount_of_points):
    points_in_circle = 0
    r = 1

    for i in range(amount_of_points):
        x_p = random.random()
        y_p = random.random()

        distance = x_p**2 + y_p**2

        if distance <= r**2:
            points_in_circle += 1

    pi = 4 * (points_in_circle / amount_of_points)
    return pi
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size.
   And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independent of the input size.

Note: We could of also solved this using a full circle with `random.uniform`.

EXERCISE: APPROXIMATE AN INTEGRAL WITH MONTE CARLO METHODS

Think of trying to find an integral from a to b for $f(x)$ as trying to find the area under the curve of $f(x)$ from $x_1 = a$ to $x_2 = b$.

- **Area of a rectangle:** $a_r = \text{length} * \text{width}$

$$f(x) = \sin(x)$$

We want to try and approximate $\int_0^\pi f(x)\,\mathrm{d}x$. So what we are saying is: "We want the area under the curve from $x_1 = 0$ to $x_2 = \pi$ for the curve $f(x)$."

We know that $-1 \leq \sin(x) \leq 1$, but since we're dealing with the first quadrant ($x \in [0, \pi]$), we will only consider positive values: $0 \leq \sin(x) \leq 1$.

To use Monte Carlo methods in geometry, we need the proportion of points in a shape inside another shape, so let's consider a rectangle around the curve.

Let's consider the rectangle:

- **width**: $w = x_2 - x_1 = \pi - 0 = \pi$.
- **height**: $h = 1$ (since $0 \leq \sin(x) \leq 1$)

The area of the rectangle would be $a_r = w * h = \pi$.

Now, let's generate n amount of random points with random x and y's, and check if they are under(and in this case inside) the curve or not.

A random point $P(x_p, y_p)$ within the rectangle is only going to be under $f(x)$ if: $y_p \leq f(x_p)$.

Let's call the area of the curve: a_c from now on.

Because the proportion of points inside the curve approximates the proportion of the curve's area relative to the rectangle's area. Then we can derive this formula:

$$\frac{\text{points in the curve}}{\text{points in the rectangle}} \approx \frac{\text{area of the curve}}{\text{area of the rectangle}}$$

$$\frac{\text{points in the curve}}{\text{points in the rectangle}} \approx \frac{a_c}{a_r}$$

And since we want the area of the curve (a_c):

$$a_c \approx \frac{\text{points in the curve}}{\text{points in the rectangle}} * a_r$$

Here are the visuals: https://www.desmos.com/calculator/um9twf3ven

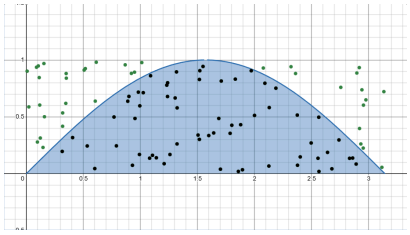


Figure 30: $n = 100$

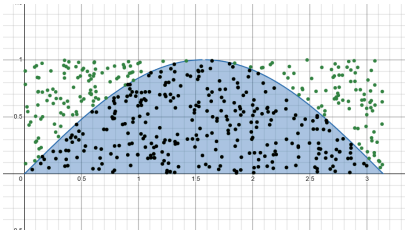


Figure 31: $n = 500$

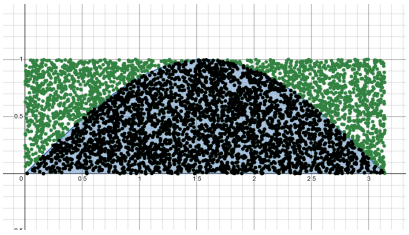


Figure 32: $n = 5000$

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - First, since we are using `sin` and `random`, we need to import the `random` and `math` libraries.
   - Then, we need to define the function `f(x)`.
   - After that we must make a function for our solution that accepts the parameter: `n`, which is the number of points to be generated randomly.
   - We need an accumulator variable to register how many points went *in the curve*, we will call it `points_in_curve`.
   - Then we could define our rectangle and get it's area.
   - Then we loop `range(n)`.
   - For each iteration, we will generate a random y between 0 and 1, and a random x between 0 and π (We will do that my multiplying the random number between 0 and 1 by π).
   - Then we will use $y \leq f(x)$ to check if the point is in the curve.
   - If it is in the curve, we will increment the `points_in_curve` accumulator.
   - Then, we use the derived formula $\frac{\text{points in the curve}}{\text{points in the rectangle}} * \text{rectangle area}$ and return the result to approximate the integral.

2.
```python
import math
import random

def f(x):
    return math.sin(x)
```

```python
def solution(n):
    points_in_curve = 0

    rectangle_height = 1
    rectangle_width = math.pi
    rectangle_area = rectangle_height * rectangle_width

    for i in range(n):
        x = random.random() * rectangle_width
        y = random.random() * rectangle_height

        if y <= f(x):
            points_in_curve += 1

    return (points_in_curve / n) * rectangle_area
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size.
   And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independent of the input size.

EXERCISE: APPROXIMATE THE VOLUME OF AN ELLIPSOID USING MONTE CARLO METHODS

An ellipsoid is a 3-D shape which is the analogue of an ellipse.

- **Formula for an ellipsoid centered** $(0, 0, 0)$: $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$, where a represents the x-stretch, b the y-stretch, and c the z-stretch.
- **Point** $P(x_p, y_p, z_p)$ **inside an ellipsoid**: $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} \leq 1$
- **Point** $P(x_p, y_p, z_p)$ **outside an ellipsoid**: $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} > 1$
- **Volume of a cuboid**: $a_c = \text{width} * \text{height} * \text{length}$, where l is the length, w is the width, and h is the height.

For this exercise, we will take $a = 2.5$ units, $b = 1.5$ units, $c = 1$ unit for our ellipsoid.

To use Monte Carlo methods in geometry, we need the proportion of points in a shape inside another shape, so let's consider a cuboid(3-D rectangle) around the ellipsoid.

Now let's figure out the cuboid:
- **width**: $w = 2 * a = 2 * 2.5 = 5$
- **length**: $l = 2 * b = 2 * 1.5 = 3$
- **height**: $h = 2 * c = 2 * 1 = 2$

Since stretch is in both directions, we multiply every stretch factor by 2.

This time we will not constrain our self with values from 0 to 1, and will simulate it fully.

Now let's figure out the volume of the cuboid:

$$a_c = w * h * l = 5 * 3 * 2 = 30 \text{ units}$$

Let's now consider a_e as the volume of the ellipsoid, and compute $\frac{a_e}{a_c}$:

$$\frac{a_e}{a_c} = \frac{a_e}{30}$$

Now, we generate n amounts of points with random x, y's and z's, and check if they are in the ellipsoid to derive the formula:

$$\frac{\text{points in the ellipsoid}}{\text{points in the cuboid}} \approx \frac{a_e}{30}$$

$$a_e \approx 30 * \frac{\text{points in ellipsoid}}{\text{points in cuboid}}$$

Here are the visuals: https://www.desmos.com/3d/4fqfrfahxc

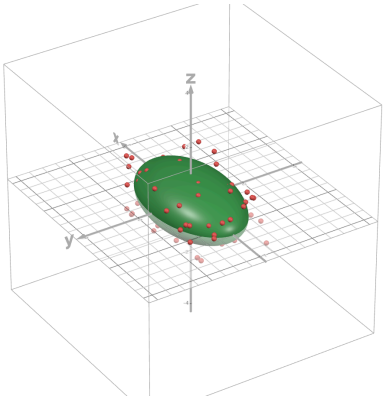


Figure 33: $n = 100$

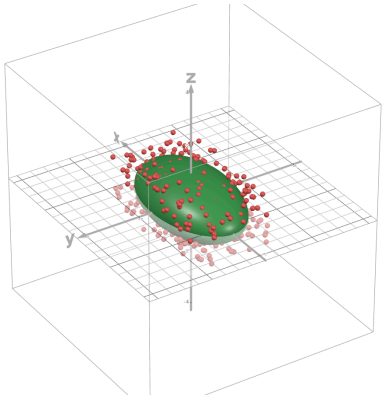


Figure 34: $n = 500$

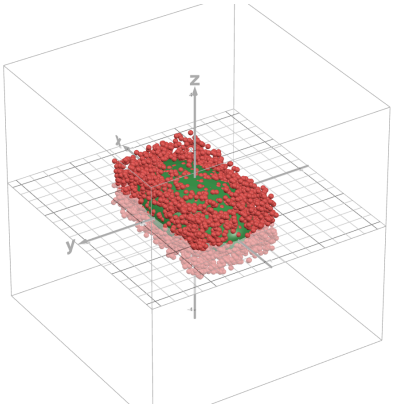


Figure 35: $n = 5000$

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - First we import the random library to get the random values.
   - Then we make the function that accepts the parameter n, the number of random points to be generated.
   - We make an accumulator variable `points_in_ellipsoid`, that counts how many points are in the ellipsoid.
   - We will register variables for the x-stretch, y-stretch and z-stretch so that we can use them later in the formula.
   - We will then loop over `range(n)`.
   - For each loop, we will generate a random x, y, and z value between $]-a, a[$, $]-b, b[$ and $]-c, c[$ respectively, using `random.uniform`.
   - We will then apply the formula $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} \leq 1$ to check if the point is inside the ellipsoid.
   - If the point is inside the ellipsoid, we will increment the `points_in_ellipsoid` by 1.
   - Then, since the formula $30 * \frac{\text{points in ellipsoid}}{\text{points in cuboid}}$ is already given to us, we can just apply it and return the result.
2.

```python
import random

def solution(n):
    points_in_ellipsoid = 0

    a = 2.5
    b = 1.5
    c = 1

    for i in range(n):
        x = random.uniform(-a, a)
        y = random.uniform(-b, b)
        z = random.uniform(-c, c)

        if ((x**2) / (a**2)) + ((y**2) / (b**2)) + ((z**2) / (c**2)) <= 1:
            points_in_ellipsoid += 1

    return (points_in_ellipsoid / n) * 30
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size.
   And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independent of the input size.

EXERCISE: ESTIMATE THE AREA OF A SUPERELLIPSE USING MONTE CARLO METHODS

Equation for a superellipse: $|\frac{x}{a}|^n + |\frac{y}{b}|^n = 1$, where a is the x-stretch, and b is the y-stretch.
Point $P(x_p, y_p)$ inside the superellipse: $|\frac{x}{a}|^n + |\frac{y}{b}|^n \leq 1$
Point $P(x_p, y_p)$ outside the superellipse: $|\frac{x}{a}|^n + |\frac{y}{b}|^n > 1$
Area of a rectangle: $a_r = \text{length} * \text{width}$

Take $a = 1$, $b = 2$ and $n = 1.5$ for the superellipse.

Find the area of the superellipse using monte carlo methods.

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1. Before writing our strategy, we need to derive the formula for the area of the superellipse in terms of the points in the superellipse, points in the rectangle and area in the rectangle.

We need to inscribe the superellipse in the rectangle, so we can get these dimensions:
- **width**: $w = 2a = 2(1) = 2$
- **height**: $h = 2b = 2(2) = 4$

We now need the area of that rectangle: $a_r = \text{width} * \text{height} = 2 * 4 = 8$

Now we can consider the area of the superellipse as a_s and compute: $\frac{a_s}{a_r} = \frac{a_s}{8}$.

Now we can derive the equation:

$$\frac{a_s}{8} \approx \frac{\text{points in superellipse}}{\text{points in the rectangle}}$$

$$a_s \approx \frac{\text{points in the superellipse}}{\text{points in the rectangle}} * 8$$

Now we can make the strategy:

- First we need to import the `math` and `random` libraries since we need their functionalities.
- Then, we make the `solution` function, that accepts `number_of_points`, which is the number of random points to be generated.
- Then, we make our accumulator variable `points_in_superellipse`.
- Then we can define our constants a, b, and n.
- Then we can loop over `range(number_of_points)` and generate a random x_p and y_p each time.
- Then we can compute the formula $|\frac{x}{a}|^n + |\frac{y}{b}|^n \leq 1$, for each point.
- If the point is inside the superellipse, then we can increment the accumulator, otherwise do nothing.
- In the end, we can return our derived formula $\frac{\text{points in the superellipse}}{\text{points in the rectangle}} * 8$.

2.
```python
import random
import math

def solution(number_of_points):
    points_in_superellipse = 0

    a = 1
    b = 2
    n = 1.5
    rectangle_area = 8

    for i in range(number_of_points):
        x_p = random.uniform(-a, a)
        y_p = random.uniform(-b, b)

        distance = (math.fabs(x_p/a) ** n) + (math.fabs(y_p/b) ** n)

        if distance <= 1:
            points_in_superellipse += 1

    return (points_in_superellipse / number_of_points) * rectangle_area
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size.
   And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independent of the input size.

8. Exams

8.1. Section 1 : Choose the correct answer (6 points)

8.1.1. 1-) Random Access Memory is volatile storage because:

A) It loses all stored data instantly when electrical power is disconnected
B) Its speed changes unpredictably depending on software load
C) Data can only be written once and never modified
D) It is highly fragile and physically breaks easily

8.1.2. 2-) What is the primary role of the Central Processing Unit:

A) To permanently save files when the machine loses power
B) To connect the computer to local networks and routers
C) To display monitor output
D) To execute sequences of instructions

8.1.3. 3-) Python is interpreted because:

A) A compiler translates the code into instructions the CPU can run
B) A program called the interpreter executes the code line by line
C) It bypasses the CPU entirely and executes code directly inside the computer's RAM

8.2. Section 2 (3 points):

1-) Convert 01000011 to base-10
2-) Compare arguments and parameters in a function
3-) Compare the integer and a float data-type
4-) Copy and complete the truth table

a	b	a **AND** b	a **OR** b	**NOT** a	**NOT** b
False	False				
True	False				
True	True				
False	True				

8.3. Section 3: Identify the pieces of code (3 points)

For each of the functions below, determine/identify the following:

- The name of the function
- The parameters they accept
- The procedure (steps) the function does
- The output (result) of the procedure

Function A:
`def divide(a,b):`
`  return a / b`
	Function B:
	`def greet(name):`
	`  greeting = "Good morning " + name`
	`  return greeting`
		Function C:
		`def subtract_two(number):`
		`  return number - 2`

Function D:	**Function E**:	**Function F**:

```
ef give_last_element(num_list, list_length):
    return num_list[list_length - 1]
```

```
def does_name_exist(names_list, target_name):
    does_exist = False

    for name in names_list:
        if name == target_name:
            does_exist = True

    return does_exist
```

```
def is_adult(age):
    if age >= 18:
        return True
    else:
        return False
```

8.4. Section 4: From The Text (8 points)

General Instruction Types

Instructions are grouped into ordered lists that, when taken as a whole, tell the different parts of the computer how to work together to perform a specific task, like grayscaling an image or playing a media file. These ordered lists of instructions are called *programs*, and they consist of a few basic types of instructions.

In modern RISC microprocessors, the act of moving data between memory and the registers is under the explicit control of the code stream, or program. So if a programmer wants to add two numbers that are located in main memory and then store the result back in main memory, he or she must write a list of instructions (a program) to tell the computer exactly what to do. The program must consist of:

- a `load` instruction to move the two numbers from memory into the registers
- an `add` instruction to tell the ALU to add the two numbers
- a `store` instruction to tell the computer to place the result of the addition back into memory, overwriting whatever was previously there

These operations fall into two main categories:

Arithmetic instructions
These instructions tell the ALU to perform an arithmetic calculation (for example, `add`, `sub`, `mul`, `div`).

Memory-access instructions
These instructions tell the parts of the processor that deal with main memory to move data from and to main memory (for example, `load` and `store`).

Figure 36: Inside The Machine - Jon Stokes

1-) State 4 examples of arithmetic instructions?
2-) State 2 memory-access instructions?
3-) What does a load, add, and store instruction tell the computer to do?
4-) What does the text refer to an ordered list of instructions as?

8.5. Section 5: Bonus

1-) What is a deterministic function? State an example of a function that is deterministic, and why?
2-) Write 4 examples of comparison operators
3-) The following function takes an n-length number list, express the total amount of arithmetic operations done by the function(algorithm) in terms of n
(Note: Each addition, subtraction, division and multiplication is an arithmetic operation and the only operation that should be accounted for)

```python
def calculate_each_element(num_list, list_length):
    for number in num_list:
        first_calculation = number + list_length
        second_calculation = first_calculation - number
        third_calculation = second_calculation / 2
        print(third_calculation)
    return "Finished calculating"
```

4-) 4.1-) What is the main purpose of time complexity analysis?
4.2-) What is the theoretical minimum time
complexity of finding the i-th largest element

Introduction to Time and Space Complexity (+ different notations)

Time Complexity is a notation/ analysis that is used to determine how the number of steps in an algorithm increase with the increase in input size. Similarly, we analyze the space consumption of an algorithm for different operations.

This comes in the analysis of Computing Problems where the theoretical minimum time complexity is defined. Some Computing Problems are difficult due to which minimum time complexity is not defined.

For example, it is known that the Problem of finding the i-th largest element has a theoretical minimum time complexity of O(N). Common approaches may have O(N logN) time complexity.

One of the most used notations is known as Big-O notation.

For example, if an algorithm has a Time Complexity Big-O of O(N^2), then the number of steps is of the order of N^2 where N is the number of data.

Note that the number of steps is not exactly N^2. The actual number of steps may be 4 * N^2 + N/3 but only the dominant term without constant factors is considered.